Pharmacy Management System Report

Student Name: Prabhnoor Singh

Student Number: GH1038228

Module Code: B103 Databases & Big Data

Date: 08.04.2026

GITHUB LINK----- https://github.com/Prabhnoor545/GH1038228-pharmacy-management-database.git

1.Introduction 1
2.System Design 2
2.1 Overview of Design Approach 3
2.2 Entity Description 4
Patient 4
Doctor 4
Staff 4
Supplier 5
Medicine 5
Prescription 6
Prescription_Item 6
Sale 7
Sale_Item 7
3.Implementation 8
4. Results 9
4.1 Basic Data Retrieval 9
4.2 CRUD Operations 9
Create Operation 10
Update Operation 10
Delete Operation 10
4.3 Join Queries 10
Medicines in Each Prescription 11
Sales with Staff Names 12
4.4 Aggregation Queries 12
Total Sales Amount 12
Total Quantity Sold for Each Medicine 12
4.5 Business Logic Queries 13
Low Stock Medicines 13
5.Challenges and Solutions 13
6.Conclusion 14
6.Future Work 14

1.Introduction

In healthcare today pharmacies play a very crucial role by ensuring that patients receive the correct medication and that too in atimely manner. So as pharmacies deal with a high volume of data on a regular basis which includes everything from prescriptions, patient records, medicine inventory and transactions managing this data manually can easily lead to small errors piling up and turning into a huge problem.So the use of a structured relational database system is very essential for maintaining accuracy and improving the operational efficiency.

The main focus of this project is on the design and implementation of a Pharmaceutical Management Database using SQL with the main objective being to create a system which has a well structured database and can store, organize and manage data related to pharmaceutical operations. The system aims to simulate real world pharmacy operations .For example a patient may visit a doctor and receive a prescription that contains multiple medicines and these medicines are stored in the pharmacies inventory and are supplied by various number of different suppliers .when the patient makes a purchase the transaction is recorded as a sale and is handled further by a staff member and each of these processes involve relationships between different data entitites which must be carefully modeled in the database.

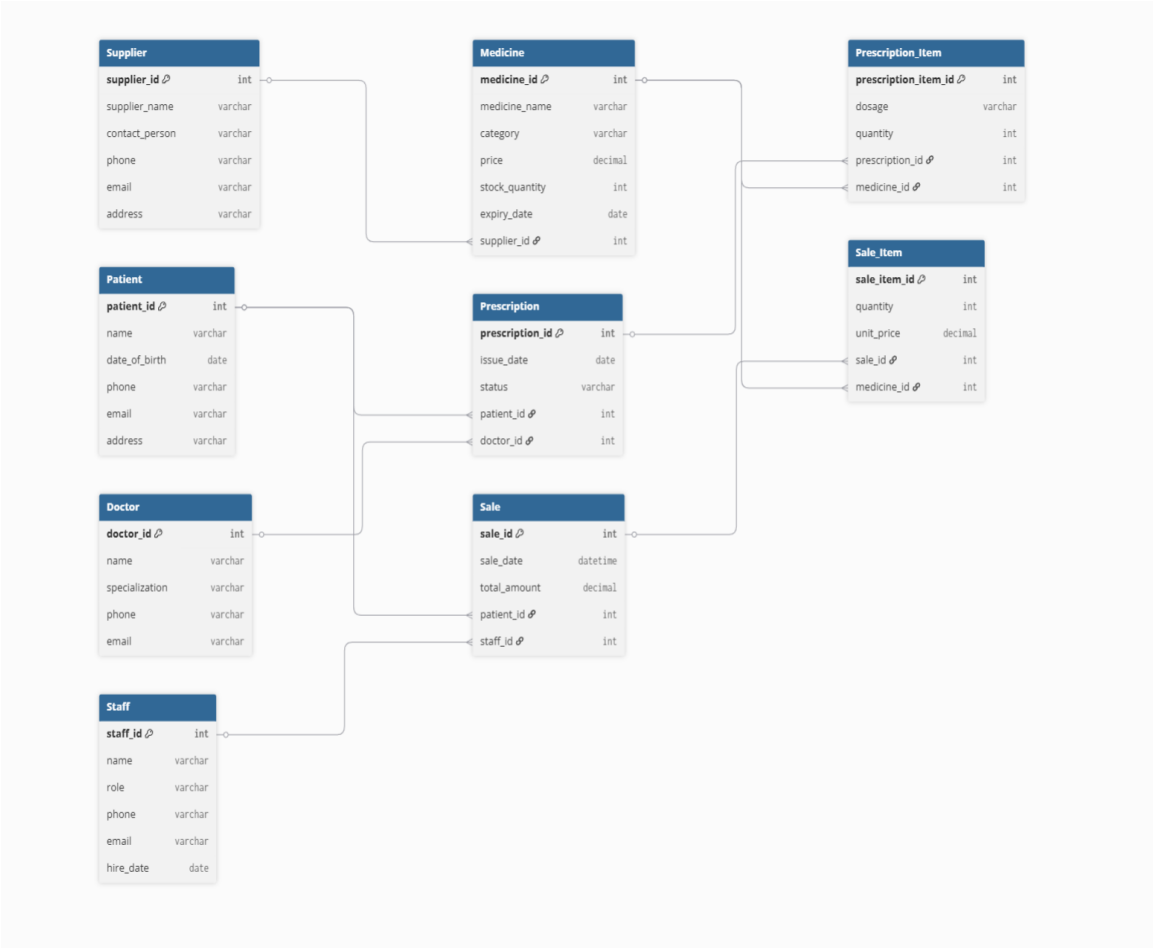
In addition to basic data storage, the system also allows for data analysis using SQL queries. These include retrieving information using SELECT queries, updating records, deleting data, performing joins to combine multiple tables, and using aggregation functions to analyze trends such as total sales or most sold medicines.

Another important aspect of this project is the application of various design principles like normalization and the use of constraints. Normalization makes sure that the database is free from redidndancy and that data is stored efficiently. Constraints such as primary keys, foreign keys, and check conditions help maintain data integrity.

Overall, this project demonstrates how a relational database system can be designed and implemented to solve real-world problems in pharmacy management. It highlights the importance of structured data organization, efficient querying, and maintaining consistency across interconnected data.

2.System Design

The system design is based on the Entity Relationship or ER model which is a very good visual representation of the databases core structure and it also helps identify the main entities,their attributes and relationships between them.This step is very crucial as a well-designed ER model leads to a clean and efficient database implementation.



Above is the ER Diagram also attached in the github repository as a pdf

2.1 Overview of Design Approach

The design of the pharmacy database was created to a similar approach taken to a real world pharmaceutical database in which each entity in the database represents a real-world object or concept, such as a patient, doctor, or medicine. These entities are connected through relationships that reflect how they interact in a pharmacy setting.

The design ensures that

a)Each entity has a unique identifier (primary key)
b) Relationships between entities are clearly defined
c) Data redundancy is minimized

d) The system can be easily extended in the future

2.2 Entity Description

Patient

The Patient entity stores information about people who receive prescriptions or purchase medicines. Each patient has its own patient ID.

Attributes

Patient ID (Primary Key)
Name
Date of Birth
Phone Number
Email Address
Address

This entity is important because it connects to both prescriptions and sales. A patient can have multiple prescriptions over time and may also make multiple purchases.

Doctor

The Doctor entity stores information about medical professionals who issue prescriptions. Each doctor has its own doctor ID.

Attributes:

Doctor ID (Primary Key)
Name
Specialization
Phone Number
Email Address

A doctor can issue multiple prescriptions to different patients. This creates a one-to-many relationship between Doctor and Prescription.

Staff

The Staff entity represents pharmacy employees who handle transactions and operations. Each staff member has its own staff ID.

Attributes:

Staff ID (Primary Key)
Name
Role
Phone Number
Email Address
Hire Date

This entity is linked to the Sale table, as each sale is handled by a staff member.

Supplier

The Supplier entity stores information about companies that supply medicines to the pharmacy.

Attributes:

Supplier ID (Primary Key)
Supplier Name
Contact Person
Phone Number
Email Address
Address

A supplier can provide multiple medicines, forming a one-to-many relationship with the Medicine entity.

Medicine

The Medicine entity represents the products available in the pharmacy.

Attributes:

Medicine ID (Primary Key)
Medicine Name

Category
Price
Stock Quantity
Expiry Date
Supplier ID (Foreign Key)

Each medicine is associated with one supplier, but a supplier can provide multiple medicines. This relationship makes sure that inventory can be traced back to its source.

Prescription

The Prescription entity represents a medical prescription issued by a doctor to a patient. It acts as a central component of the system, linking patients, doctors, and medicines together.

Attributes:

- Prescription ID (Primary Key)
- Issue Date
- Status
- Patient ID (Foreign Key)
- Doctor ID (Foreign Key)

Each prescription belongs to one patient and is issued by one doctor. However, a patient can have multiple prescriptions over time, and a doctor can issue prescriptions to multiple patients. This results in two one-to-many relationships:

- Patient → Prescription
- Doctor → Prescription

The inclusion of a status field allows the system to track the progress of prescriptions, such as whether they are pending, completed, or cancelled. This adds practical value to the database by enabling filtering and monitoring.

Prescription_Item

The Prescription_Item entity is a junction table that resolves the relationship between Prescription and Medicine.

Attributes:

Prescription Item ID (Primary Key)
Dosage

Quantity
Prescription ID (Foreign Key)
Medicine ID (Foreign Key)

In real world scenarios, a single prescription may contain multiple medicines, and a single medicine may appear in multiple prescriptions.

Sale

The Sale entity is supposed to represent as a transaction in which medicines are sold to a customer.

Attributes:

Sale ID (Primary Key)
Sale Date
Total Amount
Patient ID (Foreign Key, nullable)
Staff ID (Foreign Key)

Each sale is handled by a staff member and may optionally be linked to a patient. The patient ID is allowed to be NULL to support walk-in customers who are not registered in the system. This is an important design decision that improves the flexibility and realism of the system.

A single patient can have multiple sales, and a staff member can handle multiple transactions.

Sale_Item

The Sale_Item entity is another junction table that resolves the relationship between Sale and Medicine.

Attributes:

Sale Item ID (Primary Key)
Quantity
Unit Price
Sale ID (Foreign Key)
Medicine ID (Foreign Key)

A single sale can include multiple medicines, and a single medicine can appear in multiple sales. The Sale_Item table allows this relationship to be properly represented while also storing additional details such as quantity and price at the time of sale.

This design reflects how real-world billing systems operate, where each transaction contains multiple line items.

3.Implementation

As discussed above in the previous part, the Pharmacy Management Database was implemented using SQL based on the Entity-Relationship design presented. All entities from the diagram were transformed into tables using the "CREATE TABLE" statement. There were mainly three considerations in implementing the schema:

1. Creating a proper Schema.
2. Preserving the relationships using keys
3. Constraints to enforce business rules

The final schema comprised of nine tables: Supplier, Patient, Doctor, Staff, Medicine, Prescription, Prescription_Item, Sale, and Sale_Item. The tables were created in a logical order where parent tables (those that will be referred by others) were created before child tables (those with foreign keys pointing to other tables). For instance, the Supplier table came before the Medicine table as the latter has a foreign key to the former.

In each table, Primary Keys were included for unique identification. Also, foreign keys were used in some tables to preserve the relationships among tables as per the ER-diagram. For instance, Prescription had a foreign key for both Patient and Doctor, whereas Sale had a foreign key for Staff and an optional foreign key for Patient.

Constraints were included to enforce the business rules of the pharmacy. Mandatory fields (such as names, dates, and prices) were marked as such while check constraints ensured data is valid as per the rules. For example, negative quantities should not be allowed for medicines as it is impossible. Prescription status could only contain values of Pending, Completed, or Cancelled.

Once the schema has been created, sample data was inserted in the tables. It was necessary to insert some data as we needed data to execute various queries in our project. The order in which tables were inserted was the same logical order as described above. If done otherwise, then there would have been foreign key errors when we were inserting the rows due to incorrect relational order.

4. Results

The results section shows how the database works in practice by using SQL queries. These queries were written to test the database structure, show that relationships work correctly, and prove that the system can support meaningful pharmacy related operations. The results have everything from simple retrieval, CRUD operations, joins, aggregations, to business logic queries.

4.1 Basic Data Retrieval

(Note * an online compiler was used known as https://www.programiz.com/ but all the queries can individually be verified)

To display all patients:

patient_id	name	date_of_birth	phone	email	address
1	Rahul Singh	2000-05-10	9000000001	rahul@email.com	Berlin
2	Amanpreet Kaur	1998-08-15	9000000002	aman@email.com	Berlin
3	Vikram Patel	1995-02-20	9000000003	vikram@email.com	Berlin
4	Sneha Sharma	2001-11-25	9000000004	sneha@email.com	Berlin
5	Arjun Mehta	1999-07-30	9000000005	arjun@email.com	Berlin

To display all medicines:

medicine_id	medicine_name	category	price	stock_quantity	expiry_date	supplier_id
1	Paracetamol	Painkiller	2.5	100	2026-12-31	1
2	Amoxicillin	Antibiotic	5	80	2026-10-15	2
3	Cetirizine	Allergy	3.2	60	2026-08-20	3
4	Ibuprofen	Painkiller	4	120	2027-01-10	1
5	Vitamin C	Supplement	6.5	90	2026-11-05	2

4.2 CRUD Operations

CRUD stands for Create, Read, Update, and Delete. These operations are important as they show that the database is not only able to store information but also modify and remove it when necessary.

Create Operation

A new patient was inserted to test whether the system could add a record successfully.

SELECT * FROM Patient WHERE patient_id = 6;

Then verified with: `SELECT * FROM Patient WHERE patient_id = 6;`

patient_id	name	date_of_birth	phone	email	address
6	Riya Kapoor	2002-03-18	9000000006	riya@email.com	Berlin

Update Operation

The stock quantity of one medicine was updated to test whether existing records could be modified by using -

UPDATE Medicine SET stock_quantity = 75 WHERE medicine_id = 2;

Then verified with - SELECT * FROM Medicine WHERE medicine_id = 2;

medicine_id	medicine_name	category	price	stock_quantity	expiry_date	supplier_id
2	Amoxicillin	Antibiotic	5	75	2026-10-15	2

Delete Operation

The newly inserted patient was deleted to confirm that records can also be removed by using-

DELETE FROM Patient WHERE patient_id = 6;

And verified with - SELECT * FROM Patient WHERE patient_id = 6;

4.3 Join Queries

The most important part of the results section is showing that relationships between tables actually work.

This query shows each prescription together with the related patient and doctor names.

SELECT p.prescription_id, p.issue_date, p.status, pa.name AS patient_name, d.name AS doctor_name FROM Prescription p JOIN Patient pa ON p.patient_id = pa.patient_id JOIN Doctor d ON p.doctor_id = d.doctor_id;

prescription_id	issue_date	status	patient_name	doctor_name
1	2026-01-10	Pending	Rahul Singh	Dr. Rajesh Kumar
2	2026-01-15	Completed	Amanpreet Kaur	Dr. Priya Singh
3	2026-01-20	Completed	Vikram Patel	Dr. Rajesh Kumar
4	2026-02-01	Pending	Sneha Sharma	Dr. Ankit Verma
5	2026-02-05	Cancelled	Arjun Mehta	Dr. Priya Singh

This result demonstrates that foreign keys between preccriptions,patient and doctor were implemented correctly

Medicines in Each Prescription

This query shows which medicines belong to each prescription, together with dosage and quantity.

SELECT p.prescription_id, pa.name AS patient_name, m.medicine_name, pi.dosage, pi.quantity

FROM Prescription_Item pi JOIN Prescription p ON pi.prescription_id = p.prescription_id

JOIN Patient pa ON p.patient_id = pa.patient_id JOIN Medicine m ON pi.medicine_id = m.medicine_id ORDER BY p.prescription_id;

prescription_id	patient_name	medicine_name	dosage	quantity
1	Rahul Singh	Paracetamol	2 times a day	10
1	Rahul Singh	Amoxicillin	1 time a day	5
2	Amanpreet Kaur	Cetirizine	3 times a day	7
3	Vikram Patel	Paracetamol	2 times a day	8
4	Sneha Sharma	Ibuprofen	1 time a day	6
5	Arjun Mehta	Vitamin C	2 times a day	9

Sales with Staff Names

This query shows which staff member handled each sale.

SELECT s.sale_id, s.sale_date, s.total_amount,

st.name AS staff_name FROM Sale s JOIN Staff st ON s.staff_id = st.staff_id ORDER BY s.sale_id;

sale_id	sale_date	total_amount	staff_name
1	2026-02-10 10:30:00	25	Prabhnoor Singh
2	2026-02-11 12:00:00	15.5	Karan Malhotra
3	2026-02-12 14:20:00	40	Simran Kaur
4	2026-02-13 16:45:00	18	Prabhnoor Singh
5	2026-02-14 18:10:00	22.75	Karan Malhotra

4.4 Aggregation Queries

Aggregation queries were used to analyze the stored data and generate useful summaries.

Total Sales Amount

This query calculates the total value of all recorded sales.

SELECT SUM(total_amount) AS total_sales_amount FROM Sale;

total_sales_amount
121.25

Total Quantity Sold for Each Medicine

This query groups sales data by medicine to show how many units of each medicine were sold.

SELECT m.medicine_name, SUM(si.quantity) AS total_quantity_sold FROM Sale_Item si JOIN Medicine m ON si.medicine_id = m.medicine_id GROUP BY m.medicine_nameORDER BY total_quantity_sold DESC;

medicine_name	total_quantity_sold
Paracetamol	4
Cetirizine	3
Ibuprofen	2
Vitamin C	1
Amoxicillin	1

4.5 Business Logic Queries

To make the system more practical, business-oriented queries were also included.

Low Stock Medicines

This query identifies medicines that may need restocking.

SELECT medicine_id, medicine_name, stock_quantity FROM Medicine WHERE stock_quantity < 80;

medicine_id	medicine_name	stock_quantity
2	Amoxicillin	75
3	Cetirizine	60

5.Challenges and Solutions

Many problems came up in the process of implementing and testing the ER model. The first problem that occurred was failure of foreign key constraints during the insertion of records due to improper order of data insertion. This was solved by arranging the order in which the records were inserted and using explicit IDs to avoid confusion.

The next problem that we faced was that of the many-to-many relationship between the tables Prescriptions and Medicines, and Sales and Medicines. At first, it would have been impractical to insert multiple medicines into a single record of a prescription or sale table. But this problem could be solved by creating new junction tables such as Prescription Item and Sale Item, which provided the needed many-to-many relationship and additional attributes as well.

Another problem that occurred in the testing stage was that of debugging SQL queries. Many queries did not work as expected due to some problems related to joins and wrong assumptions about the IDs.

6.Conclusion

The Pharmacy Management Database successfully proves the application of relational database principles in a realistic scenario. The primary goal of the project was to develop a database schema and corresponding SQL queries that could help manage different aspects of the pharmacy operation, namely patients, doctors, staff, medicines, prescriptions, and sales. As demonstrated throughout the project, this task has been accomplished satisfactorily.

During the development of the database, several challenges were faced, such as the violation of foreign key constraints and difficulties in implementing many-to-many relationships. However, these obstacles were successfully resolved due to careful planning and appropriate order of insertion. Furthermore, the use of junction tables made it easier to implement many-to-many relationships and prevented the creation of redundant tables.

Overall, the current project shows a good understanding of relational database systems. The ability to create a well-structured database schema, implement it using SQL, and validate it via queries can be regarded as proof of this statement.

6.Future Work

Despite being functional and structured enough, the current version of the Pharmacy Management Database can still benefit from additional improvements and enhancements. Such work will significantly increase its usability, scalability, and overall effectiveness.

Firstly, it may be beneficial to implement a graphical user interface (GUI) to facilitate the interaction with the database. Currently, all actions in the system require writing and executing SQL statements, which can be challenging for some users. With the GUI, they will be able to perform various actions through buttons, menus, and forms, thus increasing their productivity and comfort.

Secondly, the integration of a payment and billing system can greatly improve the usefulness of the database. Even though the system already tracks sales transactions, there is no information about the payment method used. By creating a separate payment table, it will be possible to track cash, credit cards, and online payments in detail. This feature will give a more comprehensive view of finances and facilitate decision-making.

Moreover, the database can be extended to accommodate multiple pharmacies. At present, it is designed for a single store, but the addition of a branch or location table along with the necessary changes in relations will turn it into a universal database. For instance, the staff table will link each worker to a particular branch instead of just having general data. This step will greatly improve the scalability of the database and allow it to serve different companies with multiple locations.

Inventory management can be optimized by introducing notifications regarding stock levels and expiration dates. For example, when the quantity of a drug falls below a predetermined threshold, the system can send an alert to the pharmacist to order more. Another useful feature can be the notification about upcoming expiration dates to avoid dispensing expired medicines.

Lastly, it may be helpful to introduce more functionalities, including patient history, recommendations for medications, and prescription validation. Such capabilities will make the database smarter and provide it with additional features.

To summarize, while the current system represents a decent and functional implementation of a pharmacy database, there are various improvements that can be introduced. Through incorporating these additions, the database can evolve into a comprehensive and fully-functional system suitable for real-world implementation.